

[Edit](#)

Develop an API Client with GitHub Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo, tasked with building a simple client to interact with a third-party REST API. In this Challenge Lab, you will use GitHub Copilot to accelerate the development process. First, you will set up a Node.js project and make an initial API request to fetch data. Next, you will implement functions to parse and display the fetched API data in a user-friendly format. Finally, you will add robust error handling and refactor the code into modules for better organization.

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSCContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

📖 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Set up the project and make a basic API request

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using  **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

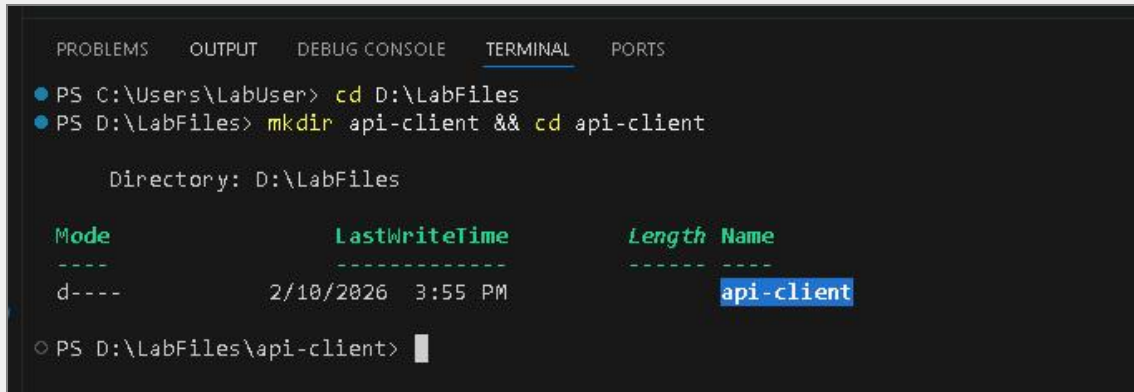
- In the  **Visual Studio Code** Terminal, create a project folder named  **api-client** in the  **D:\LabFiles** directory, and then navigate into the project folder by using the following:

 `mkdir api-client && cd api-client`

💡 Expand this hint for guidance on creating a project folder.

- On the Windows taskbar, in Search, enter **T** Visual Studio Code, and select it to open **VS Code**.
- Select **View**, and then select **Terminal** to open the integrated terminal.
- Enter **T** `cd D:\LabFiles` to navigate to the **D:\LabFiles** directory.
- Enter the commands below to create and navigate into a new project directory:

 `mkdir api-client && cd api-client`



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LabUser> cd D:\LabFiles
PS D:\LabFiles> mkdir api-client && cd api-client

Directory: D:\LabFiles

Mode                LastWriteTime         Length Name
----                -
d-----          2/10/2026   3:55 PM             api-client

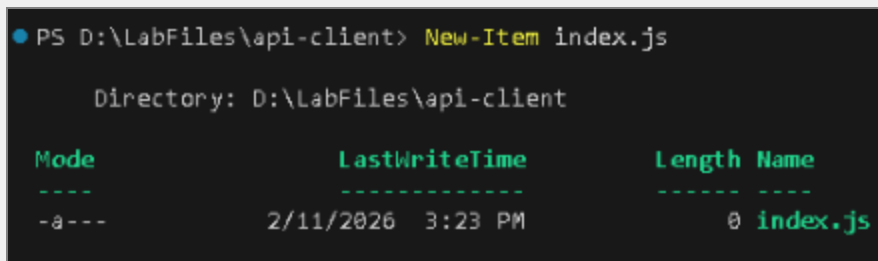
PS D:\LabFiles\api-client>
```

- Create an **T** `index.js` file in the **T** `api-client` folder by using the **T** `New-Item` command.

💡 Expand this hint for guidance on creating a file.

- In the **VS Code** Terminal, create the **T** `index.js` file in the **T** `api-client` folder by running the following command:

 `New-Item index.js`



```
PS D:\LabFiles\api-client> New-Item index.js

Directory: D:\LabFiles\api-client

Mode                LastWriteTime         Length Name
----                -
-a---          2/11/2026   3:23 PM             0 index.js
```

- In the **VS Code** Terminal, enter **T** `npm init -y` to initialize a new Node.js project with default settings.

```
PS D:\LabFiles\api-client> npm init -y
Wrote to D:\LabFiles\api-client\package.json:

{
  "name": "api-client",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS D:\LabFiles\api-client>
```

 Want to learn more? Review the documentation on [initializing a Node.js project](#).

- Install the `node-fetch` package by using the `npm install` command.

 Expand this hint for guidance on installing the `node-fetch` package. 

- In the terminal, within your project directory, enter:

 `npm install node-fetch@2`

```
PS D:\LabFiles\api-client> npm install node-fetch@2
added 4 packages, and audited 5 packages in 3s

found 0 vulnerabilities
PS D:\LabFiles\api-client>
```

 Want to learn more? Review the documentation on [the node-fetch package](#).

 The `@2` specifies version 2 of `node-fetch`, which is compatible with the CommonJS module system (`require()` function) used in this lab.

- Close the terminal window.

- Sign in to GitHub to authenticate Copilot usage in **Visual Studio Code** using `<Name>` as the username and `<GithubPW>` as the password.



Expand this hint for guidance on signing in to GitHub for Copilot.



- In **Visual Studio Code**, in the bottom right-hand corner, select **Signed out**, and then select **Sign in to use Copilot**.
- On the pop-up, select **Sign in**, and then enter `<Name>` as the username and `<GithubPW>` as the password.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.
- Select **Continue** and then **Open** to authorize **Visual Studio Code** to use GitHub Copilot.

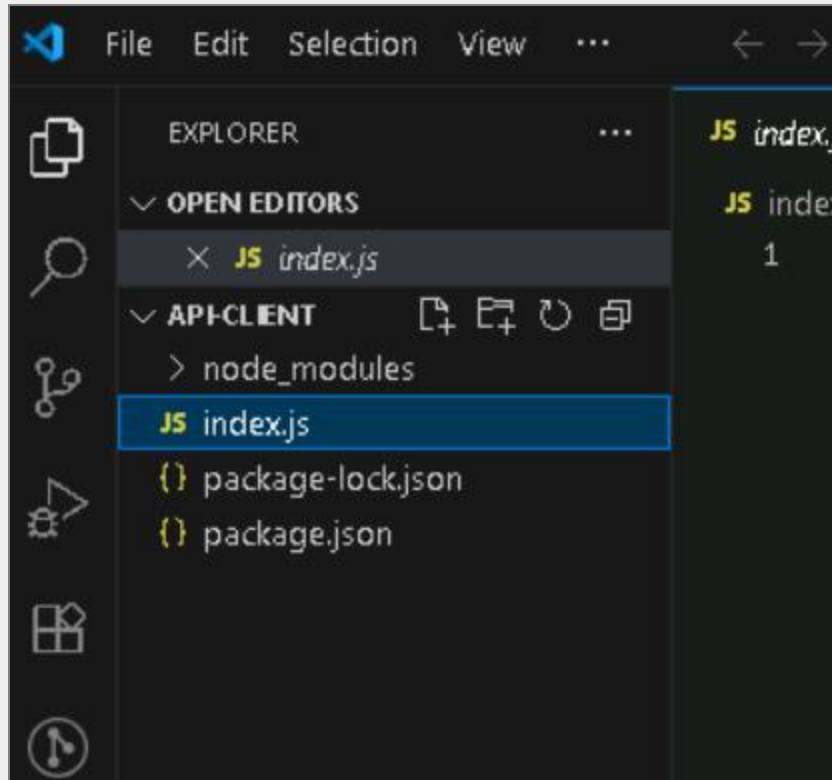


Want to learn more? Review the documentation on using a [GitHub account with Copilot](#).

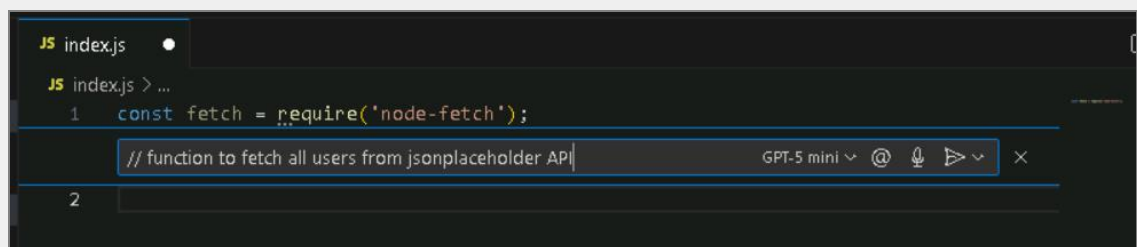
- Use GitHub Copilot to generate a `fetchAllUsers` function in the **index.js** file by using the following prompt: `// function to fetch all users from jsonplaceholder API`.

💡 Expand this hint for guidance on using GitHub Copilot to generate a `fetchAllUsers` function. ^

- In **VS Code**, in the Explorer panel, select **Open Folder**, enter `T D:\LabFiles`, press **Enter**, select the **api-client** project folder, and then select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- Open the **index.js** file.



- On the first line, enter `T const fetch = require('node-fetch');` to import the package.
- Press **Ctrl+I** to open the GitHub Copilot Inline Chat, and then enter the following comment prompt: `T // function to fetch all users from jsonplaceholder API`.



- Press **Enter**, and then wait for Copilot to generate the code.
- Accept the suggestion from GitHub Copilot, which should create an async function named **fetchAllUsers** that fetches data from <https://jsonplaceholder.typicode.com/users>.

```

JS index.js
JS index.js > fetchAllUsers
1 const fetch = require('node-fetch');

// function to fetch all users from jsonplaceholder API

⚠ You have exceeded your premium request allowance. We have automatically switched you to GPT-4.1 which is included with your plan. Enable additional paid premium requests to continue using premium models.

Ask Copilot GPT-4.1
Accept Close ↺ ↻

2 async function fetchAllUsers() {
3   const response = await fetch('https://jsonplaceholder.typicode.com/users');
4   if (!response.ok) {
5     throw new Error(`HTTP error! status: ${response.status}`);
6   }
7   const users = await response.json();
8   return users;
9 }

```

💡 Want to learn more? Review the documentation on [getting started with GitHub Copilot](#).

- ❓ If GitHub Copilot does not provide a suggestion, ensure the extension is enabled and you are logged in. You can also try rephrasing the comment to be more specific. ^

For example:

```
// Create an async function named fetchAllUsers that uses node-fetch to get all users from https://jsonplaceholder.typicode.com/users and returns the JSON response.
```

- Invoke the `fetchAllUsers` function and log the results by adding the following code (at the bottom of the file):

```
fetchAllUsers().then(users => console.log(users));
```

💡 Expand this hint for guidance on invoking the `fetchAllUsers` function and logging the results. ^

- Below the **`fetchAllUsers`** function definition, enter the following code to call the function and log its output:

```
fetchAllUsers().then(users => console.log(users));
```



Want to learn more? Review the documentation on [working with Promises in JavaScript](#).

- Save the **index.js** file by selecting **File > Save** or by pressing **Ctrl+S**.
- Execute the **index.js** script by using the **node** command in the integrated terminal.



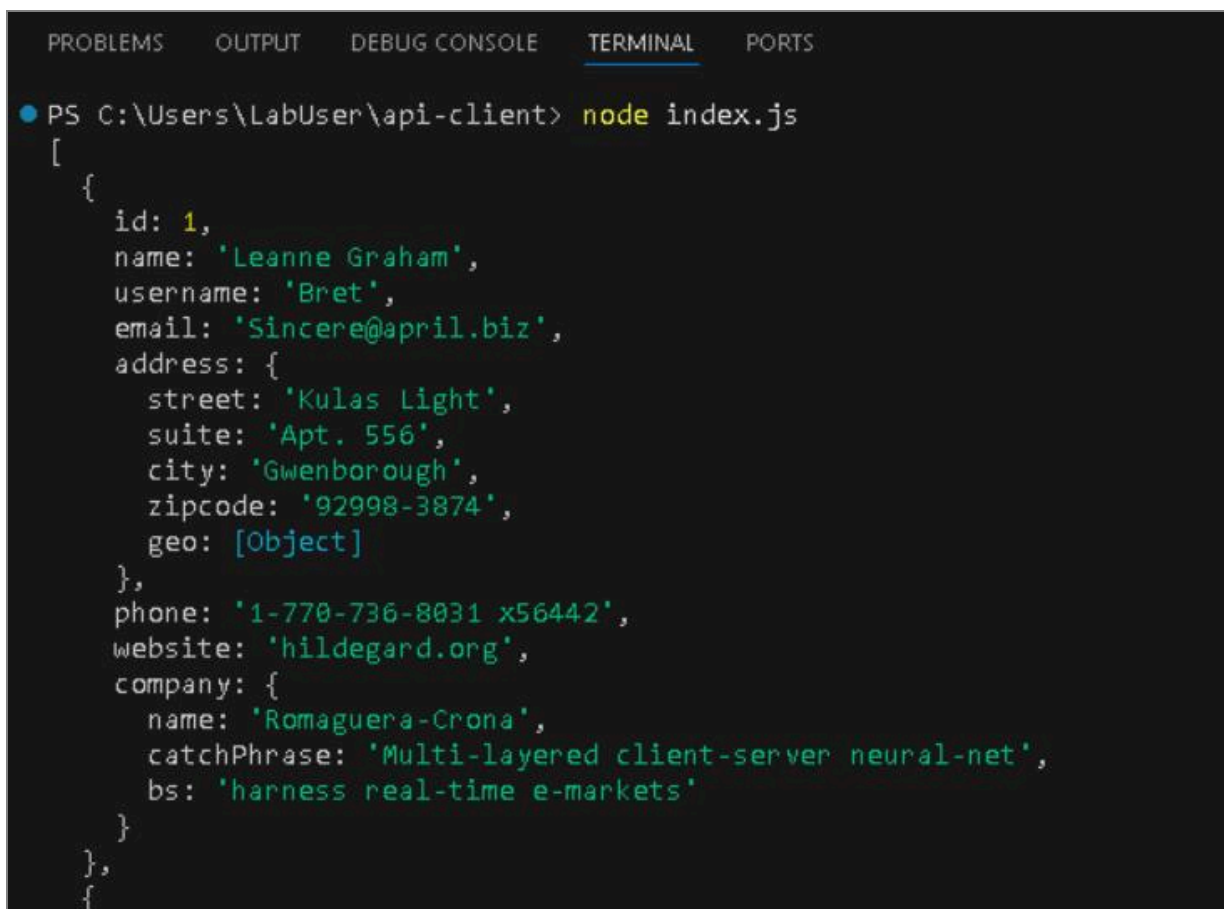
Expand this hint for guidance on running the script.



- In **VS Code**, select **Terminal**, and then select **New Terminal** to open the integrated terminal.
- Run the following command to execute the script:

 `node index.js`

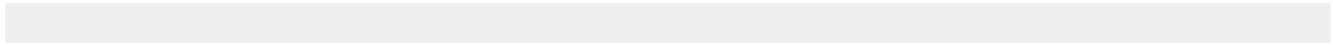
- Verify that an array of user objects is printed to the console. The output should look similar to the screenshot below:



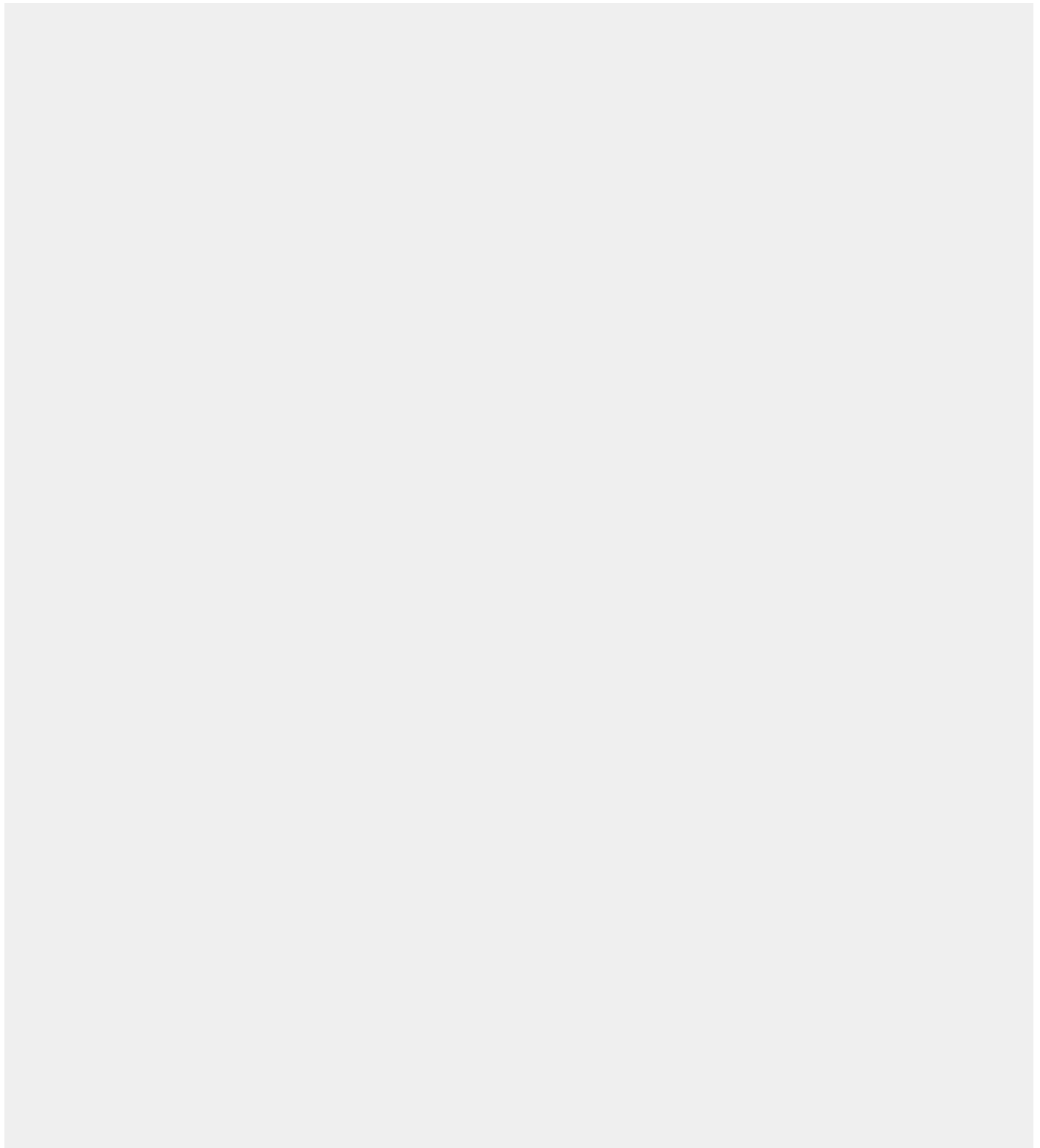
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\LabUser\api-client> node index.js
[
  {
    id: 1,
    name: 'Leanne Graham',
    username: 'Bret',
    email: 'Sincere@april.biz',
    address: {
      street: 'Kulas Light',
      suite: 'Apt. 556',
      city: 'Gwenborough',
      zipcode: '92998-3874',
      geo: [Object]
    },
    phone: '1-770-736-8031 x56442',
    website: 'hildegard.org',
    company: {
      name: 'Romaguera-Crona',
      catchPhrase: 'Multi-layered client-server neural-net',
      bs: 'harness real-time e-markets'
    }
  },
  {
```

Check your work



Verify



Implement data handling and display functions

Hints Enabled

No Yes

- Use GitHub Copilot Inline Chat to create a `T` `parseUserData` function by using the following prompt:



```
// Generate a function named parseUserData that takes an array of users  
and returns a new array with only the name and email fields. Parse user  
data and extract name and email.
```

💡 Expand this hint for guidance on using GitHub Copilot to create a `parseUserData` function. ^

- In the **index.js** file, below the **fetchAllUsers** function, enter a new comment prompt for GitHub Copilot Inline Chat (**Ctrl+I**) using the text above.

The screenshot shows a VS Code editor with a file named `index.js`. The code contains a `fetchAllUsers()` function that calls `extractNameAndEmail(users)`. Below this function, a new comment prompt is entered in the inline chat: `// generate a function named parseUserData that takes an array of users and returns a new array with only the name and email fields. parse user data and extract name and email`. The prompt is highlighted with a blue border.

- Accept the suggestion from GitHub Copilot to generate a function named `parseUserData` that takes an array of users and returns a new array with only the **name** and **email** fields.

The screenshot shows the same VS Code editor with the `parseUserData` function generated by GitHub Copilot. The function is defined as follows:

```
function parseUserData(users) {
  return users.map(user => ({
    name: user.name,
    email: user.email
  }));
}

// Example usage:
fetchAllUsers().then(users => {
  const parsedData = parseUserData(users);
});
```

The function is highlighted with a green border, and the inline chat prompt is still visible above it.

💡 Want to learn more? Review the documentation on [using JavaScript's map method](#).

- Use GitHub Copilot to generate a `displayUsers` function by using the following prompt: `// Function named displayUsers to display users in a formatted list.`

💡 Expand this hint for guidance on using GitHub Copilot to generate a `displayUsers` function.

- In the **index.js** file, enter a new comment prompt: `T // Function named displayUsers to display users in a formatted list.`

```
18
19 v fetchAllUsers().then(users => {
20   const namesAndEmails = extractNamesAndEmails(users);
21   console.log(namesAndEmails);
22 });
23
// function to display users in a formatted list
24
25
```

- Accept the suggestion from GitHub Copilot to create a function named `T displayUsers` that logs each user's name and email to the console.

```
function displayUsers(users) {
  const parsedUsers = parseUserData(users);
  parsedUsers.forEach((user, index) => {
    console.log(`${index + 1}. Name: ${user.name}, Email: ${user.email}`);
  });
}

// Example usage:
fetchAllUsers().then(displayUsers);
```

💡 Want to learn more? Review the documentation on [using GitHub Copilot for common tasks](#).

- Update the main application flow to use the new parsing and display functions by using the following code:

```
fetchAllUsers().then(parseUserData).then(displayUsers);
```

💡 Expand this hint for guidance on updating the main application flow to use the new parsing and display functions. ^

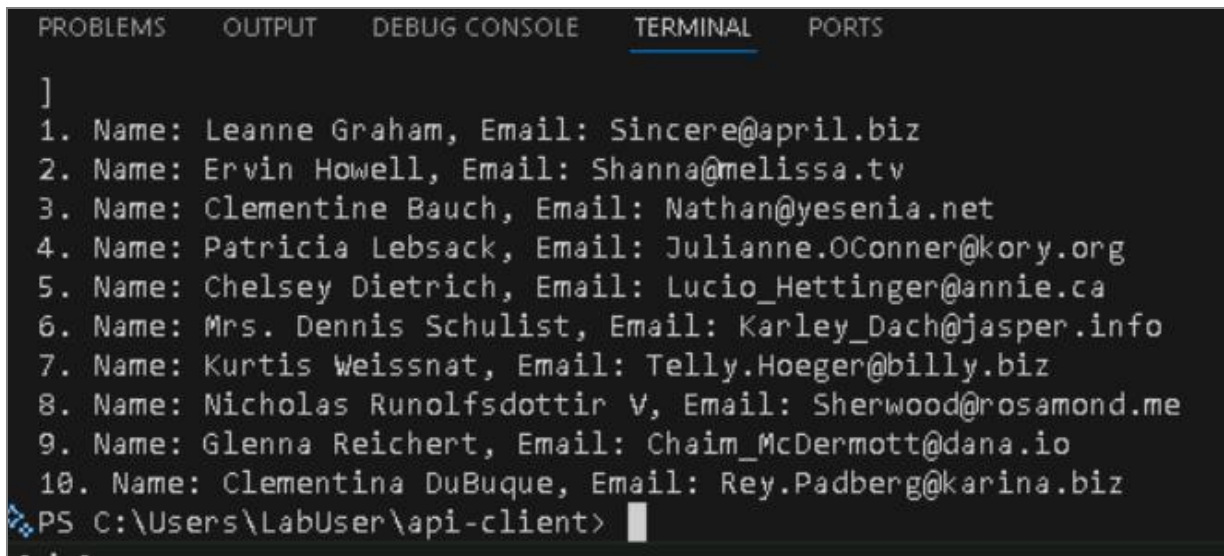
- Modify the previous call to the **fetchAllUsers** function in the **index.js** file to create a chain of promises by replacing the code with this line:

```
fetchAllUsers().then(parseUserData).then(displayUsers);
```

- If necessary, move the code to the bottom of the **index.js** file.
- Save the **index.js** file.

💡 Want to learn more? Review the documentation on [Promise chaining](#).

- In the **VS Code** Terminal, enter `node index.js` and verify that a formatted list of usernames and emails is displayed.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
]
1. Name: Leanne Graham, Email: Sincere@april.biz
2. Name: Ervin Howell, Email: Shanna@melissa.tv
3. Name: Clementine Bauch, Email: Nathan@yesenia.net
4. Name: Patricia Lebsack, Email: Julianne.OConner@kory.org
5. Name: Chelsey Dietrich, Email: Lucio_Hettinger@annie.ca
6. Name: Mrs. Dennis Schulist, Email: Karley_Dach@jasper.info
7. Name: Kurtis Weissnat, Email: Telly.Hoeger@billy.biz
8. Name: Nicholas Runolfsdottir V, Email: Sherwood@rosamond.me
9. Name: Glenna Reichert, Email: Chaim_McDermott@dana.io
10. Name: Clementina DuBuque, Email: Rey.Padberg@karina.biz
PS C:\Users\LabUser\api-client>
```

📄 This concise syntax, known as a point-free style, works because the output of each function in the `.then()` chain is passed as the single argument to the next function.

- Use GitHub Copilot to create a `fetchUserById` function by using the following prompt:

```
// An async function named fetchUserById to fetch a single user by ID as
a parameter and fetches data from the appropriate endpoint.
```

💡 Expand this hint for guidance on using GitHub Copilot to create a `fetchUserById` function. ^

- In the **index.js** file, enter a new comment prompt in the GitHub Copilot Inline Chat:

📄 `// An async function named fetchUserById to fetch a single user by ID as a parameter and fetches data from the appropriate endpoint.`

- Accept the suggestion from GitHub Copilot to generate an async function named `fetchUserById` that takes a **userId** as a parameter and fetches data from the appropriate endpoint.



The screenshot shows a GitHub Copilot inline chat window. At the top, the prompt reads: "// an async function named fetchUserId to fetch a single user by ID as a parameter and fetches data from the appropriate endpoint." Below the prompt, there are buttons for "Accept", "Close", and a refresh icon. The generated code is as follows:

```
async function fetchUserId(id) {  
  const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);  
  if (!response.ok) {  
    throw new Error(`HTTP error! status: ${response.status}`);  
  }  
  const user = await response.json();  
  return user;  
}  
  
// Example usage:  
fetchUserId(1).then(user => console.log(user));
```

💡 Want to learn more? Review the documentation on [making fetch requests](#).

Check your work

Verify

Add error handling and refactor the code

Hints Enabled

No Yes

- Add a *try...catch* block to the API fetch logic in the **fetchAllUsers** function by using the following prompt: `T // add error handling to this function.`

💡 Expand this hint for guidance on adding a try...catch block to the API fetch logic. ^

- In the **index.js** file, locate the `fetchAllUsers` function.

```
2
3 async function fetchAllUsers() {
4   const response = await fetch('https://jsonplaceholder.typicode.com/users');
5   if (!response.ok) {
6     throw new Error('HTTP error! status: ${response.status}');
7   }
8   return response.json();
9 }
```

- Use the following GitHub Copilot comment prompt: `// add error handling to this function`.

```
2 // add error handling to this function
3 async function fetchAllUsers() {
4   const response = await fetch('https://jsonplaceholder.typicode.com/users');
5   if (!response.ok) {
6     throw new Error('HTTP error! status: ${response.status}');
7   }
8   return response.json();
9 }
```

- Accept the suggestion to wrap the **fetch** and **.json()** calls within a try...catch block.

```
async function fetchAllUsers() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/user');
    if (!response.ok) {
      throw new Error('HTTP error! status: ${response.status}');
    }
    return await response.json();
  } catch (error) {
    console.error('Failed to fetch users:', error);
    return [];
  }
}
```

💡 Want to learn more? Review the documentation on [adding a try...catch block](#).

⚠ Always add error handling to network requests. APIs can be unavailable, requests can time out, or responses can be malformed. A try...catch block prevents your application from crashing.

- Configure the **catch** block in the **fetchAllUsers** function to log a user-friendly error message.

💡 Expand this hint for guidance on configuring the catch block to log a user-friendly error message. ^

- Inside the **catch** block for the `fetchAllUsers` function, ensure the code logs a message.

```
} catch (error) {  
  console.error('Failed to fetch users:', error);  
  return [];  
}
```

- The suggested code should be similar to:

```
console.error('Failed to fetch users:', error);
```

💡 Want to learn more? Review the documentation on [error handling with async/await](#).

- Repeat the previous two steps for the **fetchUserById** function, adding a similar *try...catch* block and error message by entering the `// add error handling to this function` prompt in the Copilot Inline Chat.

```
async function fetchUserById(id) {  
  try {  
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
    return await response.json();  
  } catch (error) {  
    console.error(`Failed to fetch user with ID ${id}:`, error);  
    return null;  
  }  
}
```

- Refactor the API logic into a separate module named `api.js` by using the `fetchAllUsers` and `fetchUserById` functions, including their *try...catch* blocks.

💡 Expand this hint for guidance on refactoring the API logic into a separate `api.js` module. ^

- In the Explorer window, in the **api-client** project directory, select the **New File** icon, enter `api.js`, and press **Enter**.
- In `index.js`, highlight the entire `fetchAllUsers` and `fetchUserById` functions, including their `try...catch` blocks.
- Copy and paste the selected code into the `api.js` file.
- The `api.js` file should look similar to the following screenshot:

```

2
3  async function fetchAllUsers() {
4      try {
5          const response = await fetch('https://jsonplaceholder.typicode.com/users');
6          if (!response.ok) {
7              throw new Error(`HTTP error! status: ${response.status}`);
8          }
9          return await response.json();
10     } catch (error) {
11         console.error('Failed to fetch users:', error);
12         return [];
13     }
14 }
15
16
17
18  async function fetchUserById(id) {
19      try {
20          const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
21          if (!response.ok) {
22              throw new Error(`HTTP error! status: ${response.status}`);
23          }
24          return await response.json();
25     } catch (error) {
26         console.error(`Failed to fetch user with ID ${id}:`, error);
27         return null;
28     }
29 }

```

💡 Want to learn more? Review the documentation on [creating Node.js modules](#).

- In the new `api.js` file, ensure `node-fetch` is imported at the top by adding the following code:

📄 `const fetch = require('node-fetch');`

```

JS api.js > ...
1  const fetch = require('node-fetch');
2
3  async function fetchAllUsers() {
4      try {

```

- At the bottom of the `api.js` file, add the following code to export the functions:

 `module.exports = { fetchAllUsers, fetchUserById };`

```
28     }  
29   }  
30  
31   module.exports = { fetchAllUsers, fetchUserById };
```

- In the terminal, enter `node index.js`, and then press **Enter**.
- Throw an intentional error by replacing the entire **index.js** script with the following sample code.

Expand this drop-down to view the Sample Script. 



```
const fetch = require('node-fetch');  
  
const FORCE_ERROR = true;  
  
async function fetchAllUsers() {  
  try {  
    if (FORCE_ERROR) {  
      throw new Error('Forced error for testing fetchAllUsers');  
    }  
  
    const response = await fetch('https://jsonplaceholder.typicode.com/users');  
  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
  
    return await response.json();  
  } catch (error) {  
    console.error('Failed to fetch users:', error.message);  
    throw error;  
  }  
}  
  
async function fetchUserById(id) {  
  try {  
    const response = await fetch(  
      `https://jsonplaceholder.typicode.com/users/${id}`  
    );  
  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
  
    return await response.json();  
  } catch (error) {  
    console.error(`Failed to fetch user with id ${id}:`, error.message);  
    throw error;  
  }  
}
```

```

    }
  }

  /* EXECUTE ON RUN */
  (async () => {
    await fetchAllUsers();
  })();

  module.exports = {
    fetchAllUsers,
    fetchUserById
  };

```

 Expand this hint for guidance on updating the index.js file. 

- In **index.js**, select the entire script, and then press **Delete**.
- Add the sample script supplied above.

 Want to learn more? Review the documentation on [using the require\(\) function in Node.js](#).

- Save the **index.js** file.
- In the terminal, enter **node index.js** to run the application.
- Verify that the error was caught.

```

]
PS D:\LabFiles\api-client> node index.js
PS D:\LabFiles\api-client> node index.js
Failed to fetch users: Forced error for testing fetchAllUsers
D:\LabFiles\api-client\index.js:8
  throw new Error('Forced error for testing fetchAllUsers');
        ^
Error: Forced error for testing fetchAllUsers
    at fetchAllUsers (D:\LabFiles\api-client\index.js:8:13)
    at D:\LabFiles\api-client\index.js:43:9
    at Object.<anonymous> (D:\LabFiles\api-client\index.js:44:3)
    at Module._compile (node:internal/modules/cjs/loader:1734:14)
    at Object..js (node:internal/modules/cjs/loader:1899:10)
    at Module.load (node:internal/modules/cjs/loader:1469:32)
    at Module._load (node:internal/modules/cjs/loader:1286:12)
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)
    at wrapModuleload (node:internal/modules/cjs/loader:235:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:152:5)

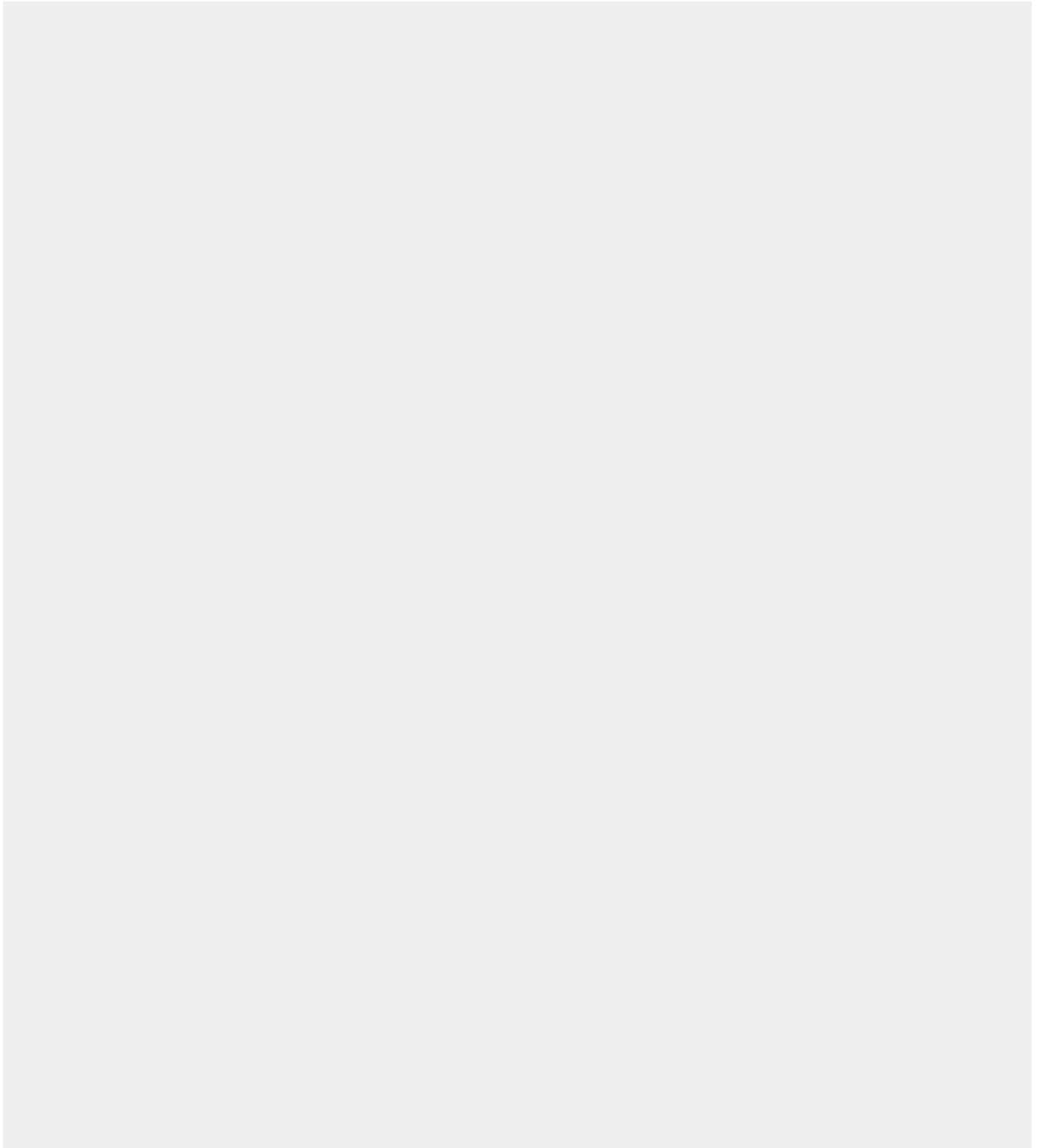
Node.js v24.2.0
PS D:\LabFiles\api-client>

```

Check your work



Verify



Summary

Congratulations, you have completed the **Develop an API Client with GitHub Copilot** Challenge Lab.

You have accomplished the following:

- Set up the project and made a basic API request.
- Implemented data handling and displayed functions.
- Added error handling and refactored the code.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]
- Refactor Python Code by Using Copilot [Guided]

- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

